

Tarea Programada #1

1. Objetivos

- Desarrollar en el estudiante la capacidad de resolver problemas en contextos modernos de programación.
- Poner en práctica los conocimientos adquiridos hasta el momento, en temas como iteración, estructuras condicionales, funciones, strings, listas, tuplas y datos numéricos.
- Utilizar la estrategia divide y vencerás para resolver un problema general, solucionando los subproblemas que lo conforman.
- Integrar todos los conocimientos adquiridos para crear un producto de software con un propósito significativo.
- Desarrollar habilidades blandas para poder trabajar correctamente en equipo: ética, autodidactas, idiomas, empatía, trabajo en equipo, etc.
- Desarrollar estrategias de investigación y uso del idioma inglés según corresponda.
- Implementar las buenas prácticas de “código limpio” y eliminación de “olores de software”.
- Usar GitHub para llevar el control de versiones.

2. Marco teórico

Archivos



Los archivos son conjuntos de datos residentes en almacenamiento secundario, como discos, que mantienen la información aun cuando se apague el computador. Los datos almacenados en archivos se conocen como datos persistentes.

Python ve cada archivo como un flujo secuencial de caracteres, donde una marca de EOF (*End of File*) determina el fin del archivo.

Con ello, usted puede crear archivos de texto de la extensión que guste, por ejemplo: **.txt**, **.csv**, **.html**, **.xls**, **.xml**, **json**, etc. Ya cada aplicación al abrirlo inteligentemente reacciona para abrirlo según corresponda.

Pero adicional a ello, se pueden guardar estructuras binarias, por ejemplo: listas, listas de listas, tuplas, diccionarios o listas de objetos.

Las posibles operaciones con archivos son: apertura del archivo, lectura, escritura y cerrado del archivo. Para mayor detalle referirse al capítulo 10 del libro *Introducción a la Programación en Python* del Profesor Jaime Solano. Adicionalmente puede consultar el siguiente [vínculo](#).

HTML 5

Antecedentes

Los orígenes de la Web

Internet no solo ha marcado uno de los más importantes avances tecnológicos del siglo XX, sino que también ha acompañado un cambio cultural de trascendencia que, en pleno siglo XXI, se mantiene en constante evolución. Pero toda historia tiene un comienzo, e Internet también lo tuvo, mucho antes de ser un fenómeno masivo.

La historia cuenta que el antecesor de Internet fue el proyecto conocido como ARPANET, una red descentralizada que algunos organismos estadounidenses utilizaron a partir de la década del sesenta. Sin embargo, el gran cambio se produciría entre fines de los ochenta y principios de los noventa, con la llegada de lo que se conoce como World Wide Web, es decir WWW, el sistema que se encarga de permitir la distribución de información mediante hipertexto.

De la mano de este cambio, comienza a popularizarse Internet en la población. Los usuarios ahora podían acceder a contenidos de la gran red, tan solo con disponer de una conexión mediante un módem y un navegador con la capacidad de interpretar contenidos de hipertexto. Esta etapa de Internet, que comprende aproximadamente desde principios de los noventa hasta el año 2003, es considerada como Web 1.0.

El concepto de este primer paradigma de la Web responde a la idea de una web “estática” o de una “sola vía”, donde el usuario es solo un “espectador” que recibe o lee contenidos, publicados por el Webmaster o dueño del sitio. Este paradigma se modificaría de manera sustancial con la llegada de la denominada Web 2.0.

Web 2.0

Los cambios en la Web no solo responden a temas tecnológicos, sino que estos van de la mano con la evolución de los hábitos de los usuarios, las

tendencias en los modos de navegación, las necesidades del mercado y hasta con aspectos culturales que también influyen en este conjunto.

La Web 2.0 representa principalmente un cambio cultural en Internet. Los usuarios, cansados de un rol pasivo, comienzan a buscar alternativas de participación. Nace una web social, donde los blogs, las redes sociales y las aplicaciones online son las estrellas. Esto ocurre a partir del año 2004.

Web 3.0

El concepto de Web 3.0 es, quizás, más complejo de definir y discutido que el caso de sus predecesores: la Web 1.0 y 2.0. Existen diversas características que la definen, entre las cuales podemos mencionar: semántica, geolocalización, Web 3D, accesibilidad desde diversos dispositivos y también inteligencia artificial.

La Web semántica, como muchas veces se define a la Web 3.0, se refiere al uso de etiquetas o bien de metadatos para otorgar un significado semántico a los elementos de la Web. Esto posibilita cierta automatización y la posibilidad de utilizar, con un mayor nivel de eficiencia, los agentes inteligentes que pueden realizar detección de contenidos.

Las características de geolocalización, muy empleadas en los equipos móviles, también han llegado a nuestro escritorio. Aunque aún pueden no ser tan precisas, las técnicas cada vez son más depuradas, y las mejoras en este campo no detienen su avance. Poder identificar a una persona, un dispositivo o cualquier elemento de manera geoespacial abre todo un mundo de posibilidades en el campo de la informática y, en especial, para todo lo referente a Realidad Aumentada.

La posibilidad de acceder desde distintos dispositivos es una realidad para una gran cantidad de usuarios y un desafío muy importante para diseñadores y desarrolladores web. Los usuarios ya no están limitados a utilizar Internet desde una computadora de escritorio, ni siquiera dependen de una laptop. Teléfonos móviles, tablets, lectores de libros electrónicos y consolas de videojuegos son solo algunas de las posibilidades que se presentan para que el usuario pueda acceder a Internet en cualquier momento y desde cualquier lugar.

W3C

El World Wide Web Consortium (W3C) es el ente o consorcio, de alcance internacional, que se encarga de crear las reglas que se utilizan como recomendaciones fundamentales para la estandarización de los principales lenguajes y tecnologías utilizados en Internet, como el caso de HTML, CSS, XML, DOM y SVG

Lenguajes de etiquetas

Los lenguajes de etiquetas, también conocidos como lenguajes de marcado o de marcas, son los que nos permiten estructurar un documento mediante el uso de etiquetas. Un ejemplo muy popular de un lenguaje de etiquetas es HTML. Algunos otros son: XML, SGML, entre otros.

HTML

HTML (HyperText Markup Language o lenguaje de marcado de hipertexto) es el lenguaje de etiquetas que funciona como una de las piedras angulares de la World Wide Web. Aunque la evolución de Internet nos ha traído muchos avances en lo que se refiere a tecnología (Web 2.0 y Web 3.0, mediante), el lenguaje de etiquetas que se popularizó en la década del noventa sigue siendo fundamental para el desarrollo web, ya que es el que comprenden e interpretan los navegadores.

HTML5

HTML5 plantea una evolución necesaria para HTML, que luego de más de una década en la versión 4.01 necesitaba, de manera imperiosa, una renovación para estar al día con las necesidades del desarrollo web actual.



En HTML5, se destacan sus características semánticas, las posibilidades multimedia que incorpora, las nuevas funciones para formulario y las características que se definen para poder integrarse con tecnologías que permitirán abrir una nueva etapa en Internet, en lo que se refiere a la arquitectura de las aplicaciones. Por estos motivos, HTML5 es considerado como uno de los motores más importantes de la Web 3.0.

Ejemplo de estructura básica de un documento en formato HTML5

```
1  <!DOCTYPE html>
2
3  <html lang="es">
4
5  <head>
6  <title>Titulo de la web</title>
7  <meta charset="utf-8" />
8  <link rel="stylesheet" href="estilos.css" />
9  <link rel="shortcut icon" href="/favicon.ico" />
10 <link rel="alternate" title="Pozolería RSS" type="applicat
11 </head>
12
13 <body>
14   <header>
15     <h1>Mi sitio web</h1>
16     <p>Mi sitio web creado en html5</p>
17   </header>
18   <section>
19     <article>
20       <h2>Titulo de contenido</h2>
21       <p>Contenido (ademas de imagenes, citas, video
22     </article>
23   </section>
24   <aside>
25     <h3>Titulo de contenido</h3>
26     <p>contenido</p>
27   </aside>
28   <footer>
29     Creado por mi el 2011
30   </footer>
31 </body>
32 </html>
```



GIT

Recuerda que, para iniciar en tu proceso de ser auto didacta, debes comprender para usar el video proporcionado de Git con el apoyo de la estudiante avanzada de carrera y el Ingeniero Gerardo con su perspectiva desde el ambiente laboral:

Pasos para utilizar GitHub: Recuerda certificarte cómo se te indicó al final de la presentación, ello es currículum para encontrar empleo.

Dedica tiempo para aprender del video que se colocó en la carpeta de tareas Programadas.



¡Por que considere!...



Método de depuración del patito de goma

El **método de depuración del patito de goma** es un término informal utilizado en [ingeniería de software](#) para describir un método de [revisión de código](#). El nombre es una referencia a una historia del libro [The Pragmatic Programmer](#) en donde un programador toma un [patito de goma](#) y revisa su código forzándose a sí mismo a explicarlo, línea por línea, al pato. ^{[1][2]} Existen otros muchos términos para esta técnica ^[cita requerida], que a menudo tienen que ver con objetos inanimados.



Muchos programadores han tenido la experiencia de explicar un problema de programación a alguien más, posiblemente a alguien que no sabe nada sobre programación, y encontrar la solución en el proceso de explicar el problema. Al comparar lo que supuestamente hace el código con lo que hace en realidad, cualquier incongruencia resulta evidente. ^[3] Usando un objeto inanimado, el programador puede tratar de lograr el mismo efecto sin tener que hablar con otra persona.

Ello promueve varios tipos de estrategias, puede googlear para aprender más al respecto.

La estrategia que se quiere promover en este curso es: [La programación en pareja](#)

La programación en pareja (pair programming, en inglés) requiere que dos programadores participen en un esfuerzo combinado de desarrollo en un sitio de trabajo. Cada miembro realiza una acción que el otro no está haciendo actualmente: Mientras que uno codifica las pruebas de unidades el otro piensa en la clase que satisfará la prueba, por ejemplo.



Programación en pareja

La persona que está haciendo la codificación se le da el nombre de controlador mientras que a la persona que está dirigiendo se le llama el navegador. Se sugiere a menudo para que a los dos socios cambien de papeles por lo menos cada media hora o después de que se haga una prueba de unidad.

Compilación

La compilación es el proceso mediante el cual un programa escrito en un lenguaje de alto nivel es transformado a una forma que puede ser ejecutada por una computadora, ya sea código máquina o una representación intermedia. Este proceso se divide en varias etapas, entre las cuales destacan el análisis léxico, análisis sintáctico, análisis semántico, optimización y generación de código. Cada una de estas fases cumple una función específica en la validación y transformación del programa fuente.

En el contexto de esta tarea, se simula una parte del proceso de compilación, específicamente el análisis léxico, donde el código fuente es procesado como texto y transformado mediante reglas definidas. Aunque no se construye un compilador completo, se aplican principios fundamentales de este proceso para manipular y reinterpretar el código.

Análisis Léxico

El análisis léxico es la primera etapa del proceso de compilación. Su función principal es leer el código fuente como una secuencia de caracteres y agruparlos en unidades significativas llamadas tokens. Este proceso elimina espacios innecesarios, identifica palabras clave, operadores, identificadores y otros elementos del lenguaje.

El analizador léxico utiliza patrones definidos, comúnmente mediante **expresiones regulares**, para reconocer estos elementos. En esta tarea, el uso de expresiones regulares

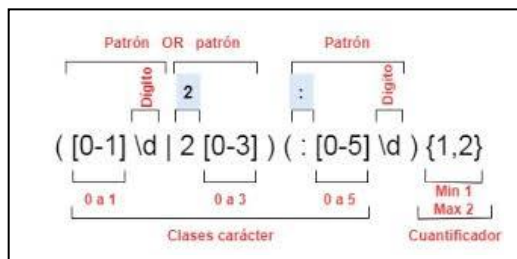
permite identificar palabras clave dentro del código Python y reemplazarlas correctamente, simulando el comportamiento de un analizador léxico simplificado.

Tokens

Un token es la unidad mínima con significado dentro de un lenguaje de programación. Los tokens representan elementos como palabras clave (if, for, while), identificadores, operadores, literales y símbolos especiales. Durante el análisis léxico, el código fuente es descompuesto en una secuencia de tokens que luego serán utilizados en etapas posteriores del proceso de compilación.

En esta tarea, los tokens son utilizados como sustitutos de palabras clave del lenguaje Python. A través de un archivo de configuración, se define una correspondencia entre keywords y nuevos tokens, lo que permite transformar el código original en una versión modificada del lenguaje.

Expresiones Regulares



Las expresiones regulares son herramientas utilizadas para definir patrones de búsqueda dentro de cadenas de texto. Son ampliamente utilizadas en el análisis léxico para identificar tokens dentro de un código fuente. Permiten realizar coincidencias precisas, como detectar palabras completas, evitar coincidencias parciales

y realizar reemplazos controlados.

En el desarrollo de esta tarea, las expresiones regulares permiten localizar palabras clave dentro del código Python utilizando patrones como límites de palabra, asegurando que los reemplazos sean correctos y no afecten otras partes del texto.

Por hacer:

En la actualidad, muchas empresas tecnológicas requieren herramientas que permitan adaptar y transformar código fuente según diferentes necesidades, como internacionalización, enseñanza o experimentación con lenguajes derivados. Esto implica procesar texto de manera estructurada, respetando la sintaxis de un lenguaje de programación y generando salidas útiles para distintos contextos.

Como parte de esta tarea programada, se requiere desarrollar un sistema capaz de traducir código Python mediante el uso de tokens personalizados. Estos tokens serán definidos en **archivos externos** y aplicados al código fuente utilizando técnicas de procesamiento de texto, incluyendo el uso de **expresiones regulares** para asegurar reemplazos correctos y controlados.

Además, el sistema no solo deberá transformar código, sino también generar reportes estructurados en distintos formatos, como **CSV y HTML**, simulando así un entorno real donde los sistemas deben producir información procesada para: análisis, auditoría o visualización.

El programa deberá trabajar directamente con archivos, permitiendo: cargar configuraciones, procesar código fuente y generar múltiples salidas. Esto implica manipulación de rutas, lectura y escritura de archivos, así como la correcta estructuración de datos.

Funcionalidades requeridas:

El sistema debe implementarse mediante un menú interactivo en consola que incluya al menos las siguientes opciones:

1. Cargar archivo de tokens

Permite leer un archivo de texto que contiene equivalencias entre palabras clave de Python y tokens personalizados. Debe pedir al usuario que digite el nombre del archivo a usar, además del método de separación usado en el archivo (por ejemplo, si usó “->”, o si usó “,”, o si usó “=”). Si se leen varios archivos de tokens y algún token se reescribe, debe indicar por medio de un print en consola que se reescribió, y conservar el reemplazo más reciente. Estos reemplazos deben ser manejados dentro del programa como **listas de tuplas**.

Ejemplos de formato:

```
int -> entero  
str -> hilera
```

```
int, entero  
str, hilera
```

```
int=entero  
str=hilera
```

2. Mostrar tokens cargados

Despliega todas las equivalencias actualmente en memoria. Debe hacer un “pretty print” de todas las equivalencias ya cargadas, de manera que el usuario pueda ver con claridad cada reemplazo.

3. Agregar o modificar tokens

Permite al usuario ingresar nuevos tokens o actualizar existentes. El usuario digitará una cadena larga con varios tokens que deben ser procesados de manera similar a la opción 1. El usuario debe especificar el separador usado en esta cadena. Debe validar duplicados, por lo que si el usuario intenta agregar un token que ya existe, debe indicársele que está realizando una actualización. Si el token no existe aún, debe indicar que se está añadiendo. Además, debe implementar alguna estrategia para cancelar esta operación.

4. Guardar tokens en archivo

Permite persistir las configuraciones actuales del programa en un archivo nuevo. Similar a la primera opción, debe solicitar al usuario que ingrese el nombre del archivo al que se quiere guardar esta configuración, además del separador que se quiere usar para almacenarlos.

5. Traducir código

Permite al usuario seleccionar un archivo cualquiera que será procesado por el siste[Capte la atención de los lectores mediante una cita importante extraída del documento o utilice este espacio para resaltar un punto clave. Para colocar el cuadro de texto en cualquier lugar de la página, solo tiene que arrastrarlo.]

ma. Este debe leerse línea por línea, y debe guardar el resultado en otro archivo, cuyo nombre y extensión de archivo será especificado por el usuario.

¿Qué cuenta como palabra a reemplazar?

Tomando este código como ejemplo:

```
def foo(i:int):  
    i="Hola"  
    return i+1
```

Se cuentan, en total, **8** palabras:

```
def foo(i:int):  
    i="Hola"  
    return i+1
```

def, foo, i, int, i, "Hola", return, i.

Sugerencia: haga los splits tomando en cuenta caracteres especiales, no solo espacios en blanco. Ignore valores numéricos.



Ejemplo de traducción

Si el archivo de tokens es el siguiente:

```
return -> devuelva  
print  -> pinte  
def     -> funcion  
while  -> mientras
```

Y el código a traducir es:

```
def factorial(n):  
    res = 1  
    while n > 1:  
        res *= n  
        n -= 1  
    return res  
  
num = int(input("Digite el número de factorial a calcular: "))  
print(factorial(num))
```

Se debe producir este resultado:

```
funcion factorial(n):  
    res = 1  
    mientras n > 1:  
        res *= n  
        n -= 1  
    devuelva res  
  
num = int(entrada("Digite el número de factorial a calcular: "))  
pintar(factorial(num))
```

Para esta sección de reemplazos, no se permite el uso del método **replace()**, ni cualquier otro método de strings que realice el reemplazo de manera automática.

6. Generar reporte CSV

Crea un archivo .csv con información de los reemplazos realizados: su palabra original, su token de reemplazo, y la cantidad de reemplazos por cada palabra original. Debe determinar una estrategia para realizar cada conteo, y debe incluir la explicación de esta estrategia y su razón de uso en la **documentación escrita**.

7. Generar reporte HTML

Genera un archivo .html que contenga:

1. Hora y fecha de generación como nombre de archivo. Debe llevar el formato **reporteHTML_DD/MM/AA-hh:mm:ss.html**
2. Título del reporte (brindado por el usuario a la hora de generación). Es el título que lleva el HTML en la pestaña donde está abierto.
3. Título principal del contenido de la página. Debe decir **Reporte de Traducción**, y debe ser un H1.
4. Subtítulo que indique la fecha y hora de generación del reporte. Debe ser un **H2**.
5. Estadísticas del proceso: duración total del procesamiento, cantidad total de reemplazos y porcentaje de palabras reemplazadas. Debe usar la etiqueta <p>.
6. Tabla de tokens: palabra original, reemplazo, y cantidad de reemplazos. Asígnele un título a cada columna de la tabla, y cada fila por medio debe mostrarse de un color diferente para mejorar la lectura. Además, el contenido de cada celda debe estar centrado.



8. Bitácora de registros

Debe crear un sistema de bitácora que permita registrar todos los eventos ocurridos en el programa durante su ejecución, desde el uso de cualquier opción del menú y la inserción de un nuevo reemplazo (debe crear un nuevo registro por cada reemplazo individual), hasta cuando el usuario ejecute la opción de salir del programa. Cada una de estas acciones debe ser agregada a una **lista de tuplas**, que debe a su vez estar guardada

constantemente en un archivo binario, llamado **bitacora.txt**. Cada registro debe ser guardado en una tupla, con el siguiente formato: ("**AAAA-MM-DD_hh:mm:ss**", "**Descripción de la acción**").

Debe además desarrollar un submenú que permita al usuario buscar registros por fecha de ejecución, o por palabra clave en la descripción.

Menú del sistema:

El programa debe ejecutarse mediante un menú iterativo en consola (sin interfaz gráfica), que debe contener al menos estas opciones:

1. Cargar tokens
2. Mostrar tokens
3. Agregar/modificar token
4. Guardar tokens
5. Traducir código
6. Generar CSV
7. Generar HTML
8. Submenú de bitácora del sistema
 - A) Acciones por día escogido
 - B) Acciones con algunas palabras clave
 - C) Salir del submenú
9. Salir

Puntos del funcionamiento a ser evaluados:

Tarea	Estudiante	Puntaje
Cargar archivo de tokens	E1	8
Mostrar tokens cargados	E2	6
Agregar o modificar tokens	E1	10
Guardar tokens en archivo	E2	10
Traducir código	E1	22
Generar reporte CSV	E2	8
Generar reporte HTML	E1	10
Insertar en bitácora	E2	10
Filtrar por día escogido	E2	8
Filtrar por palabras clave	E2	8
TOTAL PUNTOS:		100

Correctitud de la solución computacional – 75% de la nota total de la tarea

1. Manejo de archivos
2. Uso de listas y tuplas
3. Implementación de traducción
4. Uso correcto de expresiones regulares
5. Generación de CSV
6. Generación de HTML
7. Funcionamiento del menú

Análisis del código fuente – 15% de la nota total de la tarea

1. Validaciones
2. Organización del código
3. Modularización
4. Explicación del funcionamiento

REQUISITO PARA REVISAR EL PROYECTO

El requisito consiste en presentar la documentación del proyecto indicada en esta sección.

La nota de la documentación del proyecto sirve para aceptar o rechazar el proyecto: se revisan los proyectos que cumplan con este requisito en un 90% o más.

Enviar vía Tec Digital, sección EVALUACIONES, en la carpeta TP1, una carpeta comprimida (.rar, .zip, etc.) que contenga las siguientes partes:

- **Parte 1:** Una carpeta con la Documentación del proyecto (**nombre: documentaciónCódigos.PDF**).

Documentación – 10% de la nota total de la tarea

Debe incluir una carpeta con la Documentación del proyecto (nombre: documentaciónCódigos.PDF).

Portada. (10 puntos):

1. Nombre de la institución
2. Nombre de la escuela a la cual usted pertenece
3. Nombre del curso
4. Nombre del profesor(a)
5. Nombres de los Estudiantes y números de carné
6. Número de tarea programada
7. Número de semestre y año lectivo
8. Fecha de entrega
9. Estatus de la entrega (definido por el responsable de la implementación de la tarea): **[Deplorable|Regular|Buena|Muy Buena|Excelente|Superior]**.

Índice (2 puntos): debe incluir todas las secciones del documento, con hipervínculos a cada una para la navegación.

Desarrollo (55 puntos):

1. Enunciado del proyecto. Hacer referencia a este enunciado, que debe ser adjuntado a la entrega de la solución, o que referencie en qué lugar del repositorio de Git se encuentra. (1 punto)
2. Justificación de eliminación de olores (párrafo descriptivo o ejemplos, indique 5) (10 puntos)
3. Lista de validaciones estratégicas. (10 puntos)
4. Estrategia de conteo de reemplazos para las opciones de reportes. (10 puntos)
5. Reglamento de trabajo (6 puntos)
6. 2 agendas (4 puntos)
7. 2 minutas (4 puntos)
8. Cronograma (10 puntos).

Conclusiones del trabajo (23 puntos):

1. Al menos 7 problemas encontrados y soluciones a los mismos (Plantilla_Bitacora_ResoluciónProb.xls). (7 puntos)
2. Aprendizajes obtenidos, o debe hacer un listado de todas las lecciones aprendidas producto del desarrollo de la tarea programada. Las lecciones aprendidas deben ser 8 de carácter personal y 8 de carácter técnico. Es decir 4 aportados por cada uno según corresponda. (16 puntos)

Estadística de tiempos (20 puntos): un cuadro que muestre el detalle de las actividades que realizó y las horas invertidas en cada una de ellas. La estadística permite medir el esfuerzo dedicado al trabajo en términos de actividades y tiempos, lo cual puede ser una base para calcular el esfuerzo requerido en futuros trabajos. No olvide investigar sobre el **Personal Software Process (PSP)** y sus implicaciones como programador. Por ejemplo:

Actividad Realizada	Horas
Análisis de requerimientos	
Diseño de algoritmos	
Investigación de ...	
Programación	
Documentación interna	
Pruebas	
Elaboración de documentación del proyecto	
Etc.	
TOTAL	

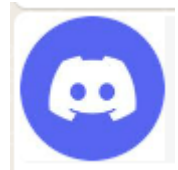
Es necesario llevar el control en paralelo del drive (por favor editar las parejas de la tarea programada e indicar un correo Gmail de cada uno) y del GitHub del avance del proyecto. Deben realizarse 3 avances cada semana obligatoriamente, es decir, al terminar la TP#1 deben hacer 18 entradas en total en el Drive (3 por persona por semana = 9, por 2 personas) y en paralelo debe coincidir con Git Hub. Esta sección no tiene nota, pero sin ello, no se revisa la TP#1.

- **Parte 2:** Una carpeta con el Programa fuente.

Condiciones generales:

Esta tarea programada se rige por las siguientes condiciones:

1. El desarrollo de la tarea es estrictamente en grupos de 2 estudiantes, si hay cambio de alguna pareja debe notificarse antes de hoy a las 12mn al correo de la profesora lsarmiento@tec.ac.cr con copia al asistente.
2. La tarea DEBE implementarse sin interfaz gráfica si no, no se revisa la tarea.
3. Debe cumplir con todo lo indicado en la sección "Puntos a ser evaluados"
4. Deberá entregarse en tiempo y forma según el plazo establecido por el profesor al momento la lectura de este documento. Se cuenta con 3 semanas a partir del día de entrega de la tarea. **Martes 12 de mayo, antes de las 11:45 pm.**
5. El lenguaje de programación a utilizar es Python v3.10.1 o superior.
6. Debe presentarse el grupo completo a defender la tarea, en caso de no asistir, tendrá nota de 0 en el valor total de la tarea.
7. Cada miembro debe realizar a conciencia la evaluación de Habilidades Blandas.
8. Sea muy ético, usted se prepara para una profesión en dónde la ética debe ser su primer pilar.
9. Deben interactuar en un discord para mejor coordinación y para la revisión de la Tarea Programada.
10. La tarea debe ser de su propia autoría o no se califica. **Se evaluará partes de la misma para ver si alguna IA influyó, si se comprobara ello, la tarea tendrá nota de 0 para ambos integrantes.**
11. La nota final de cada estudiante se asignará por distribución de esfuerzo y cumplimiento en función a las responsabilidades y la nota obtenida.



Nota: El incumplimiento de alguna condición implicará una calificación de cero.

No olvide...

Clean Code: Buenas Prácticas en el Código

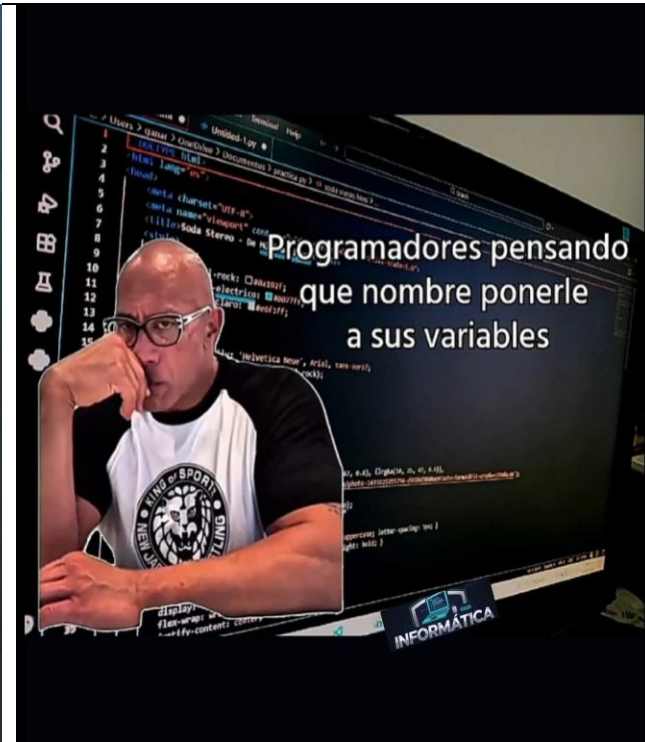
Nombres de Variables y Métodos	
❌ Mal Código	✅ Clean Code
nombres confusos	nombres descriptivos
abreviaturas	claridad y significado

Comentarios	
❌ Mal Código	✅ Clean Code
comentarios innecesarios	código autoexplicativo
explicar el "qué"	explicar el "por qué"

Funciones	
❌ Mal Código	✅ Clean Code
funciones largas	funciones pequeñas
múltiples responsabilidades	una sola responsabilidad

Formato y Legibilidad	
❌ Mal Código	✅ Clean Code
código desordenado	formato consistente
sin espacios	buen indentación

Principios Clave	
Concepto	Descripción breve
Simplicidad	código fácil de entender
Legibilidad	más importante que optimizar
Mantenibilidad	fácil de modificar



Errores que están frenando a los Programadores

Fallas más Comunes	Cómo Solucionarlos
Código Desordenado	Organiza y comenta tu código
Errores de Sintaxis	Revisa la sintaxis con regularidad
Problemas de Debugging	Usa herramientas de depuración.
Mala Gestión del Tiempo	Planifica tus tareas y descansos